

PARTE I - MÚLTIPLA ESCOLHA

1. c
2. b
3. c
4. b
5. c

PARTE II - DISCURSIVAS

Questão 6 Teste automatizado é aquele script que roda para validar se uma parte do sistema está funcionando sozinha (tipo um script testando se o alerta de enchente do app do INPE é disparado na hora certa). Já a cobertura de testes é a porcentagem do código que realmente foi percorrida por esses testes. E vulnerabilidade é alguma brecha de segurança no código ou nas bibliotecas, tipo uma falha que permitiria alguém mandar um falso alerta de queimada pra população.

Questão 7 Ter 300 testes passando não significa muita coisa se a cobertura é de só 40%, porque isso quer dizer que 60% do código da aplicação nunca foi testado. Os testes podem estar rodando só em funções super simples, enquanto partes críticas do sistema podem estar cheias de bugs que só vão estourar na mão do usuário.

Questão 8 A equipe precisa focar em criar mais testes pra pasta Utils urgente. Arquivos utilitários geralmente têm funções de conversão e formatação que o sistema inteiro usa (os Controllers e Services dependem deles). Se tiver um erro na Utils que não foi testado, o sistema inteiro quebra.

Questão 9

Porque a cobertura de testes só avalia se a linha de código foi executada pelo script, e não se o teste previu todos os problemas. Você pode ter um código com alta cobertura, mas que falha em produção porque o teste não cobriu cenários de falta de internet, erro de banco de dados ou problemas com concorrência de usuários.

Questão 10 O npm test é o comando que roda a suíte de testes automatizados do projeto (ele busca o script de teste lá no package.json). Ele costuma rodar no pipeline de CI logo depois da fase de build. No projeto do INPE, ele é a garantia de

que ninguém subiu um código novo que estragou as funcionalidades de envio de alertas.

Questão 11 Não concordo. Vulnerabilidades marcadas como "High" são muito graves e podem facilitar invasões ou vazamento de dados. Como o app do INPE lida com informações públicas e de emergência ambiental, o deploy deveria ter sido bloqueado na hora até que a equipe rodasse um npm audit fix para atualizar esses pacotes.

Questão 12 Isso acontece quando a gente instala um pacote ou biblioteca criada por outras pessoas e esse código externo vem com falhas de segurança conhecidas. Por exemplo, usar um pacote de terceiros para gerar relatórios de enchentes no app do INPE, e esse pacote ter uma brecha que permite um invasor acessar o servidor do projeto.

Questão 13 Não garante. Assim como expliquei na questão 9, uma cobertura de 97% ajuda muito a evitar erros básicos, mas não impede falhas de regras de negócio, problemas na infraestrutura (como o servidor cair) ou testes que não preveem situações reais de uso do usuário.

Questão 14 O relatório serve como uma métrica concreta. O gerente pode definir uma regra (por exemplo, "nada sobe pra produção com menos de 80% de cobertura") e usar esse relatório para barrar ou liberar um deploy com muito mais segurança, em vez de decidir no "achismo".

Questão 15 A recomendação é bloquear o deploy imediatamente. Mesmo com os testes e a cobertura estando super bons, subir para produção com 5 vulnerabilidades "High" num sistema crítico de monitoramento de desastres é um risco de segurança muito alto. Primeiro corrige as dependências, depois faz o deploy.

Questão 16 Os testes são o que fazem a Integração Contínua ser segura. No CI, os desenvolvedores juntam o código de todo mundo num repositório central várias vezes ao dia. Os testes automatizados rodam em cada uma dessas integrações pra confirmar que o código do desenvolvedor "A" não quebrou o trabalho do desenvolvedor "B".

Questão 17 Porque receber dados externos da população é muito imprevisível. As pessoas podem enviar dados incompletos, com formatos errados, e durante um desastre vai ter um pico enorme de pessoas acessando ao mesmo tempo. Os testes rigorosos garantem que a API não vai travar bem na hora que a galera mais precisar relatar um incêndio.

Questão 18

- Build: Avisa se o código instalou e compilou direitinho.

- Tests: Mostra se os testes automatizados passaram ou se teve alguma falha.
- Coverage: Dá a porcentagem de código que foi testado.
- Vulnerabilities: Mostra se o projeto tem brechas de segurança vindas dos pacotes instalados.

Questão 19 A afirmação do desenvolvedor está errada. A máquina e os testes garantem apenas que o código "funciona", mas não avaliam se ele está bem escrito, limpo e com uma arquitetura fácil de dar manutenção no futuro. A revisão de código (Code Review) feita por outro humano serve para olhar a qualidade e as boas práticas, coisa que a cobertura não mede.

Questão 20 Eu olharia 4 indicadores principais antes de liberar:

1. Status do Build: pra ter certeza que a aplicação monta e roda sem dar erro fatal.
2. Testes: tem que estar 100% passando, pra garantir que as funções de emergência do INPE continuam operando.
3. Vulnerabilidades: não deixaria passar nada classificado como High ou Critical, pra não expor o sistema a ataques.
4. Cobertura (Coverage): pra garantir que o time de desenvolvimento manteve o nível de qualidade e não subiu um monte de código sem teste.